



Nolte Geoservices

UXO Target Sync Platform

Target Tracker & Data Visualization Platform
Complete Project Documentation

Project: 11-24-2736 — Wilhelmshaven Rüstersieler Seedeich

Repository: C:\Apps\feedbackapp (branch: backendtest)

Generated: 16 July 2026

Table of Contents

1. What Is This Project? (Plain-Language Overview)
2. Key Concepts & Glossary
3. System Architecture
4. Technology Stack
5. Project Folder Structure
6. The Database (Data Model)
7. The Backend Server (FastAPI)
8. API Reference
9. Coordinate Systems & Conversions
10. The Data Ingestion Pipeline
11. The Frontend Web Application (React)
12. Offline-First Design & Synchronization
13. Status Classification Logic
14. How to Run the Project
15. Configuration Reference
16. Troubleshooting
17. Known Limitations & Notes for Future Development

1. What Is This Project? (Plain-Language Overview)

This project is a **field data-collection and visualization platform** built for **Nolte Geoservices GmbH**, a German geophysics company. Its purpose is to support **UXO clearance** work — the process of finding and safely removing *unexploded ordnance* (old wartime bombs, mines, and munitions) buried in the ground before construction work begins.

The concrete use case is a real project: “**11-24-2736 — Wilhelmshaven Rüstersieler Seedeich**”, a sea-dike site in Wilhelmshaven, northern Germany. The workflow it supports looks like this:

1. **Survey phase (already done, outside this app).** Geophysicists scan the site with two instruments — **georadar** (ground-penetrating radar, GPR) and **magnetometers**. Each suspicious underground signal is recorded as an **anomaly** (also called a *target*): a point with coordinates and an estimated depth. These results arrive as CSV exports (`Radar_data.csv` , `Magnetic_data.csv`).
2. **Loading targets into the platform.** A Python script (`ingest_anomalies.py`) cleans, merges, and loads the anomalies from both instruments into a central PostgreSQL/PostGIS database. The raw CSVs contain ~2,430 magnetic rows and ~45 radar rows; after dropping rows with missing or duplicate coordinates, the live database currently holds **1,583 anomalies** and **61 feedback records**.
3. **Field investigation.** A field worker (the *collector* role) opens the web app on site, sees all targets on an interactive map, walks/digs to each one, and records what was actually found: the excavated depth, dimensions of the hole, the object found (iron bar, wire, nothing, etc.), photos, and notes. This works **even without an internet connection** — entries are stored in the browser and synced later.
4. **Office monitoring.** Back at the office, an analyst (the *dashboard* role) sees live progress: how many targets are investigated, what was found, charts comparing predicted vs. actual depths, excavated volume, and a map colored by status.

In one sentence: it is an offline-capable “ArcGIS Field Maps”-style web app plus backend, purpose-built for tracking the investigation of suspected UXO targets and syncing field findings with a central geospatial database.

2. Key Concepts & Glossary

Term	Meaning in this project
UXO	Unexploded Ordnance — buried explosive remnants (bombs, mines, shells) that must be located and removed.
Anomaly / Target / Point	A single suspicious spot detected by a survey instrument. The three words are used interchangeably in the code. Stored in the <code>anomalies</code> database table.
Georadar / GPR	Ground-Penetrating Radar — an instrument that images the shallow subsurface with radar pulses.
Magnetic / Magnetometer	An instrument that detects buried ferrous (iron-containing) metal by its magnetic field.
Feedback	The record a field worker creates after physically investigating one target (what was found, how deep, photos, notes). Stored in the <code>feedback</code> table.
VM Nr.	"Verdachtsmoment-Nummer" — a human-friendly sequential target label like <code>2736-17</code> (project short-code + running number).
target_id	A stable, unique string built from project id + coordinates, e.g. <code>11-24-2736-442972.981-5937097.795</code> . Used to derive a deterministic UUID for each anomaly.
Fundstück (German)	"Found object" — what the excavation uncovered, e.g. <i>Eisenstange</i> (iron bar), <i>Eisendraht</i> (iron wire), <i>ohne Fund</i> (nothing found).
Sohle / Sohle status (German)	"Pit floor." <i>Frei</i> = cleared to the bottom; <i>Nicht Frei</i> = could not be fully cleared (e.g. blocked by a utility pipe).
Tiefe / Länge / Breite / m³	German for depth / length / width / cubic metres — the dimensions of the excavation opening.
UTM 32N (EPSG:32632)	The metric map-projection coordinate system used by German surveyors (easting/northing in metres). The web map instead needs WGS84 latitude/longitude (EPSG:4326), so the code converts between the two.
PostGIS	A PostgreSQL extension that adds geospatial data types (<code>GEOMETRY</code>) and functions, letting GIS tools like QGIS/ArcGIS read the database directly.
Offline-first	A design where the app keeps working with no network: data lives in the browser (IndexedDB) and is pushed to the server when connectivity returns.

3. System Architecture

The system has three tiers plus a one-off data pipeline:



Why this shape?

- **The field has poor connectivity**, so the browser keeps its own full copy of all targets in IndexedDB and queues unsent work. The server is the single source of truth once syncing succeeds.
- **The office uses professional GIS tools** (QGIS, ArcGIS Pro) that talk to PostGIS directly. Database triggers make sure a point moved in QGIS updates lat/lon for the web app, and a point dragged on the web map updates the PostGIS geometry — bidirectionally.
- **One deployable unit:** the built React app is copied into `./static` and served by FastAPI itself, so a single `python server.py` serves both the API and the UI on port 8000.

4. Technology Stack

Layer	Technology	Role
Backend	Python 3.13	Language
	FastAPI + Uvicorn	Web framework and ASGI server (port 8000)
	SQLAlchemy 2 (+ psycopg 3)	ORM / database access
	GeoAlchemy2	Maps the PostGIS <code>GEOMETRY</code> column into SQLAlchemy models
	Pydantic + pydantic-settings	Request validation and <code>.env</code> configuration
Database	PostgreSQL 16 + PostGIS 3.4	Primary store, run via Docker Compose (<code>postgis/postgis:16-3.4</code>)
	pgAdmin 4	Web-based database admin UI (<code>dpape/pgadmin4</code> , port 8080), also in the compose file
	SQLite	Automatic local fallback (<code>uxo_local.db</code>) when PostgreSQL is down
Frontend	React 19 + TypeScript	UI framework
	Vite 8	Dev server and production bundler
	Leaflet + react-leaflet 5	Interactive map with switchable basemaps
	Recharts 3	Dashboard charts (bar, pie, area)
	Dexie 4	Friendly wrapper around browser IndexedDB (offline storage)
	lucide-react	Icon set
	PWA assets (<code>manifest.json</code> , <code>sw.js</code>)	Installable-app / service-worker scaffolding in <code>static/</code>
Pipeline	pandas	CSV cleaning and bulk ingestion (<code>ingest_anomalies.py</code>)
Tooling	Docker Compose, Node.js 22 (bundled in <code>node/</code>), ESLint	Infrastructure and code quality

5. Project Folder Structure

```
C:\Apps\feedbackapp\
├─ server.py           ← FastAPI backend: all API endpoints + static file serving
├─ models.py           ← SQLAlchemy table definitions (Project, Anomaly, Feedback)
├─ database.py         ← DB connection, table init, PostGIS triggers, UTM↔lat/lon math
├─ config.py           ← Settings (reads DATABASE_URL from .env)
├─ ingest_anomalies.py ← CSV → PostgreSQL bulk-ingestion pipeline (pandas)
├─ test_utm.py         ← Standalone sanity check for the UTM conversion math
├─ install_node.py     ← Helper that downloads a portable Node.js into ./node
├─ requirements.txt    ← Python dependencies
├─ docker-compose.yml  ← Spins up PostgreSQL/PostGIS + pgAdmin containers
├─ .env                ← DATABASE_URL (not committed)
├─ Radar_data.csv      ← Raw georadar survey export (~45 targets)
├─ Magnetic_data.csv   ← Raw magnetometer survey export (~2,400 targets)
├─ nolte_local.db      ← A local SQLite database file (fallback/dev data)
├─ static\             ← Production build of the React app (served at "/")
├─ │ index.html, assets\, logo.png, manifest.json, sw.js ...
├─ frontend\           ← React + TypeScript source code
├─ │ package.json, vite.config.ts, tsconfig*.json
├─ │ └─ src\
├─ │ │ └─ App.tsx           ← Root component: login, routing, sync logic
├─ │ │ └─ db\indexedDb.ts   ← Dexie schema + status-resolution helper
├─ │ │ └─ components\
├─ │ │ │ └─ FieldMap.tsx    ← Leaflet map, markers, basemaps, CSV/PDF export
├─ │ │ │ └─ FeedbackForm.tsx ← Investigation entry form incl. camera capture
├─ │ │ │ └─ Dashboard.tsx   ← Analyst dashboard with charts and KPIs
├─ │ │ │ └─ ImportExport.tsx ← CSV paste-import and demo-data seeding
```

Note: the `node\` folder contains two full portable Node.js distributions (v22.11, v22.13) installed by `install_node.py` so the frontend can be built on machines without Node installed. It is tooling, not application code.

6. The Database (Data Model)

Three tables, defined in `models.py`, with a one-to-many chain: a **project** has many **anomalies**, and each anomaly can have many **feedback** records (though in practice the app shows only the latest one).

6.1 projects

Column	Type	Description
<code>project_id</code>	VARCHAR(50), PK	e.g. 11-24-2736
<code>project_name</code>	VARCHAR(255)	e.g. "Wilhemshaven Rüstersieler Seedeich"
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	Audit timestamps

6.2 anomalies — the survey targets

Column	Type	Description
id	VARCHAR(36), PK	Deterministic UUIDv5 derived from <code>target_id</code> (see box below)
project_id	FK → projects	Cascade delete
instrument	VARCHAR(50)	georadar or magnetic
geom	PostGIS POINT, SRID 32632	Geospatial geometry for GIS tools
easting / northing	FLOAT	UTM 32N metres (survey coordinates)
latitude / longitude	FLOAT	WGS84 degrees (for the web map)
vm_nr	VARCHAR(50)	Sequential label, e.g. 2736-17
category	VARCHAR(50)	Currently always <code>Kat-1</code>
layer	VARCHAR(255)	Source survey layer, e.g. <code>Stoerkoeper Magnetik Nord</code>
status	VARCHAR(50)	<code>pending</code> → <code>investigated</code>
target_id	VARCHAR(100), UNIQUE	<code>{project}-{easting}-{northing}</code>
evaluated_depth	FLOAT	Depth predicted by the survey instrument (m)

Stable IDs — a deliberate design choice. Anomaly IDs are not random. They are `uuid5(NAMESPACE_DNS, target_id)`, i.e. a hash of the project + coordinates. Re-running the ingestion or the seed always produces the *same* IDs, so feedback recorded against a target never becomes orphaned when the target list is reloaded. (Commit `b39c85b` introduced this.)

6.3 feedback — field investigation results

Column	Type	Description
id	VARCHAR(36), PK	Created client-side so offline entries have an identity
anomaly_id	FK → anomalies	Which target this report belongs to
visited	BOOLEAN	Was the target physically visited
visit_date	TIMESTAMP	When it was logged (drives “latest wins” conflict resolution)
tief	FLOAT	Actual excavated depth (m)
laenge / breite / m_cube	FLOAT	Opening length, width (m) and volume (m³)
fundstueck	VARCHAR(255)	What was found (“Eisenteil”, “ohne Fund”, ...)
sohle_status	VARCHAR(50)	<code>Frei</code> / <code>Nicht Frei</code>
bilder_n	INTEGER	Number of photos
photos	TEXT	JSON array of base64-encoded images
notes / other	TEXT	Free-text remarks / “Sonstige” description
investigator / investigator_username	VARCHAR(100)	Who logged it
target_id	VARCHAR(100)	Denormalized copy for traceability

6.4 The spatial synchronization trigger

`database.py` → `init_db()` installs a PL/pgSQL trigger (`sync_anomaly_geom_and_coordinates`) that fires before every insert/update on `anomalies` and keeps all three coordinate representations consistent, whichever one was edited:

- **Case 1 — geom changed** (e.g. a point moved in QGIS/ArcGIS Pro): recompute easting/northing and, via `ST_Transform(..., 4326)`, latitude/longitude.

- **Case 2 — easting/northing changed** (e.g. edited numerically): rebuild `geom` and lat/lon.
- **Case 3 — latitude/longitude changed** (e.g. a marker dragged on the web map): rebuild `geom` in UTM 32N and easting/northing.

7. The Backend Server (FastAPI)

`server.py` is the whole backend — about 520 lines. On startup it:

1. Calls `init_db()` : connects to the database, enables PostGIS, creates the three tables if missing, and installs the sync trigger. If PostgreSQL cannot be reached within 5 seconds it **falls back to a local SQLite file** (`uxo_local.db`) so development still works without Docker.
2. Creates the FastAPI app titled “Nolte Geoservices UXO Target Sync Platform”.
3. Enables fully-open CORS (any origin/method/header) — convenient for development, to be tightened for production.
4. Mounts `./static` at `/`, so the compiled React app is served by the same process.

Request/response shapes are declared as Pydantic models: `FeedbackCreate` (one field report), `PointCreate` (one imported target), `PointUpdate` (moved coordinates), and `SyncPayload` (a batch of feedback + point moves).

8. API Reference

Method & Path	Purpose	Behaviour
GET <code>/api/points</code>	Fetch every target with its latest feedback	For each anomaly, finds the newest <code>feedback</code> row (by <code>visit_date</code>), derives a display status (see §13), parses the photos JSON, and returns one combined object per target. This is what the frontend caches into IndexedDB.
POST <code>/api/sync</code>	Push queued field data to the server	Accepts <code>{feedback: [...], point_updates: [...]}</code> . For each feedback item: skips it (with a warning) if its parent anomaly doesn't exist; inserts it if new; updates it only if the incoming <code>logged_at</code> is newer than the stored <code>visit_date</code> (last-write-wins conflict resolution). Marks the anomaly <code>investigated</code> . Applies any coordinate updates. On DB failure it rolls back and returns HTTP 400 with a hint to seed the database. Responds with counts plus a fresh full point list so the client can refresh its cache in the same round-trip.
GET <code>/api/stats</code>	Progress summary	Returns total/visited/unvisited counts, completion percentage, and a distribution over the statuses <code>clear</code> / <code>scrap</code> / <code>uxo</code> / <code>false_alarm</code> / <code>unvisited</code> .
POST <code>/api/points/import</code>	Import targets from the UI's CSV tool	Ensures project <code>11-24-2736</code> exists, converts each row's UTM coordinates to lat/lon, assigns the next <code>vm_nr</code> sequence numbers and deterministic UUIDs, and inserts them as <code>georadar</code> anomalies.
POST <code>/api/seed</code>	Reset + load demo data	Destructive: wipes feedback, anomalies, and projects; re-creates the project; inserts 29 hard-coded real targets transcribed from an Excel report; then pre-fills 4 of them with example feedback (a cleared iron bar, scrap metal, a false alarm, and a “PMN-2 landmine” UXO case) so all marker colours/statuses are demonstrable.
GET <code>/</code>	The web app	Serves <code>static/index.html</code> (the built React app), or a JSON status message if <code>static/</code> is absent.

9. Coordinate Systems & Conversions

Two coordinate systems coexist, and the project converts between them in *three* places:

System	Used by	Example
UTM Zone 32N (EPSG:32632) — metres	Surveyors, CSVs, PostGIS geometry, excavation reports	E 442 972.981, N 5 937 097.795
WGS84 (EPSG:4326) — degrees	Leaflet web map, GPS	53.57° N, 8.14° E

- **Python** — `database.py` implements `utm32n_to_latlon()` and `latlon_to_utm32n()` from scratch using the standard Transverse-Mercator series expansion on the WGS84 ellipsoid (no external geodesy library). `test_utm.py` is a small standalone script for sanity-checking the math.
- **TypeScript** — the same formulas are ported to JS: `utm32nToLatLonJS()` in `ImportExport.tsx` (imports) and `latLonToUtm32nJS()` in `FeedbackForm.tsx` (converting a dragged marker's new lat/lon back to easting/northing while offline).
- **SQL** — the PostGIS trigger uses `ST_Transform` for the same conversions server-side (the authoritative version).

10. The Data Ingestion Pipeline (`ingest_anomalies.py`)

This script performs the initial bulk load of survey results into PostgreSQL. Steps:

1. **Read** `Radar_data.csv` (georadar) and `Magnetic_data.csv` (magnetometer) with pandas, auto-detecting whether the separator is `;` or `,`.
2. **Tag & merge**: add an `instrument` discriminator column to each set, concatenate them, and rename the German survey columns (`Rechtswert` → `easting`, `Hochwert` → `northing`, `Tiefe [m]` → `evaluated_depth`).
3. **Clean**: drop rows missing coordinates; fix a known systematic coordinate error in the “Stoerkoerper Magnetik Sued 2” layer (easting short by 397,000 m, northing off by +21,000 m); drop duplicate coordinates.
4. **Enrich**: assign `project_id`, `status='pending'`, `category='Kat-1'`, sequential `vm_nr` labels, coordinate-derived `target_id` strings, deterministic UUIDv5 ids, and WKT point geometry text.
5. **Load: drops and rebuilds** the three tables (destructive!), re-creates them via `init_db()` (which also reinstalls the trigger), seeds the project row, bulk-stages the DataFrame into a temp table with `to_sql`, then inserts into `anomalies` converting WKT to PostGIS geometry with `ST_GeomFromText(..., 32632)`.

Warning: both `ingest_anomalies.py` and `POST /api/seed` erase existing anomalies/feedback before loading. Never run them against a database holding real field data you want to keep.

11. The Frontend Web Application (React)

The UI is a single-page app with two “roles” selected at login, both sharing one collapsible sidebar (theme toggle, navigation, sync button, online/offline indicator, user avatar, sign-out).

11.1 Login / landing page

A full-screen landing page styled after ArcGIS Field Maps: company logo, project description, and a Sign-in modal with login, sign-up, and forgot-password views. **Authentication is demo-grade only:** any username/password is accepted; profiles and roles are stored in browser `localStorage` (`nolte_users_profiles`), and demo accounts `collector` and `dashboard / admin` map to the two roles. There is no server-side auth at all.

11.2 Collector view (“Field App”)

- **Target list panel:** search box, filters by VM number, instrument, project and status, and a scrollable card list showing each target's VM number, instrument/layer, finding, and evaluated depth.
- **Map (`FieldMap.tsx`):** a Leaflet map with four switchable basemaps (dark, light, satellite, streets), circle markers coloured by status (red = pending, green = investigated), popups, an auto-zoom controller that fits the filtered points, and a toolbar (zoom, home). A selected investigated target can be exported as a **single-target CSV** or a **printable PDF report** (via a print window).
- **Feedback form (`FeedbackForm.tsx`):** opened by selecting a target. Records opening length/width/depth, computed volume, Fundstück (found object, from the standard German list), Sohle status (Frei / Nicht Frei), notes, and photos — either uploaded (client-side compressed) or captured live with the device **camera** (`getUserMedia` + `canvas` snapshot). It also supports an **edit-location mode** where the marker becomes draggable; the new position is converted back to UTM and queued as a point update.
- **Import tab (`ImportExport.tsx`):** paste CSV text to import GPR targets (online → `POST /api/points/import` ; offline → stored locally), plus a button to trigger the demo seed endpoint.

11.3 Dashboard view (analyst)

A full-screen map with floating translucent panels (`Dashboard.tsx`) showing:

- KPI tiles: total targets, investigated vs. pending, project count, total excavated volume vs. a 250 m³ target capacity.
- **Fundstück chart** — bar chart of what has been found, by category.
- **Sohle split chart** — Frei vs. Nicht Frei per finding type.
- **Depth comparison** — predicted (`evaluated_depth`) vs. actual (`tief`) depth distribution, plus a mean depth-error KPI.
- The same status/instrument filters, wired to the shared map.

12. Offline-First Design & Synchronization

This is the heart of the app. The browser database (`indexedDb.ts` , Dexie database name `NoIteFieldDb`) has three tables:

Table	Contents
<code>points</code>	The full local mirror of all targets (+ their latest feedback), refreshed from the server whenever online.
<code>pendingFeedback</code>	Outbound queue: feedback entries created while offline (or not yet accepted by the server).
<code>pendingPointUpdates</code>	Outbound queue: coordinate corrections from dragged markers.

The synchronization lifecycle

1. **On login (online):** `fetchFromServer()` pulls `GET /api/points` and rewrites the local `points` table.
2. **Recording feedback:** `handleSaveFeedback()` always writes to the pending queues and optimistically updates the local point — the UI turns the marker green immediately, network or not.
3. **Connectivity monitoring:** the app listens to the browser's `online` / `offline` events and shows a toast + a Wi-Fi indicator in the sidebar.
4. **Auto-sync:** whenever the app is online, logged in, and the pending count > 0, `handleSync()` fires automatically (also available as a manual sidebar button with a badge showing the queue size). It POSTs the queues to `/api/sync`, clears them on success, and replaces the local cache with the authoritative point list returned by the server.
5. **Conflict handling:** the server applies *last-write-wins* per feedback id, keyed on the `logged_at` timestamp, and silently skips feedback whose target no longer exists (logging a warning).

13. Status Classification Logic

A target's display status is **derived**, not stored. The same rule set exists in Python (`GET /api/points`) and TypeScript (`getResolvedStatus()`), evaluated top-down on the latest feedback:

Order	Rule	Status	Meaning / colour
0	No feedback, or not visited	<code>unvisited</code>	Red marker — still to do
1	Fundstück = "ohne Fund" (nothing found)	<code>false_alarm</code>	The instrument signal had no physical cause
2	Notes/other text contains "uxo", "mine", "bomb", "munition", or "pmn"	<code>uxo</code>	Live ordnance — highest priority
3	Sohle status = "Nicht Frei"	<code>scrap</code>	Something found, pit not fully cleared
4	Otherwise	<code>clear</code>	Investigated and resolved (green marker)

14. How to Run the Project

Prerequisites

- Python 3.13 with `pip`
- Docker Desktop (for PostgreSQL/PostGIS) — optional thanks to the SQLite fallback
- Node.js 22 (or run `python install_node.py` to fetch a portable copy into `./node`)

Step 1 — Start the database (and pgAdmin)

```
cd C:\Apps\feedbackapp
docker compose up -d
# starts two containers:
#   feedback_postgres_db   → PostgreSQL 16 + PostGIS 3.4 on port 5432
#   nolte-pgadmin          → pgAdmin 4 web UI on http://localhost:8080
```

pgAdmin login: `admin@nolte-geoservices.com` / `admin`. Inside pgAdmin, register a server with host `db` (the compose service name), port 5432, user/password `postgres` / `postgres`.

Step 2 — Configure

Create/verify `.env` in the project root:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/nolte_geoservices
```

Step 3 — Install Python dependencies and start the backend

```
pip install -r requirements.txt
pip install pandas           # needed only for the ingestion script
python server.py             # serves API + web app on http://localhost:8000
```

Step 4 — Load data (choose one; skip if the Docker volume already holds data)

```
# Option A: full survey ingestion (~1,583 targets after cleaning) – DESTRUCTIVE
python ingest_anomalies.py

# Option B: small demo set (29 targets, 4 pre-investigated) – DESTRUCTIVE
curl -X POST http://localhost:8000/api/seed
```

Step 5 — Open the app

Browse to **`http://localhost:8000`**, click *Sign in*, and use demo account `collector` (field view) or `dashboard` (analyst view) with any password.

Frontend development mode (optional)

```
cd frontend
npm install
npm run dev      # Vite dev server with hot reload (API still expected on :8000)
npm run build    # production build → copy dist/ contents into ../static
```

15. Configuration Reference

Setting	Where	Default	Notes
DATABASE_URL	<code>.env</code> / environment	<code>postgresql://postgres:postgres@localhost:5432/nolte_geoservices</code>	<code>postgresql://</code> and <code>postgresql+psycopg2://</code> prefixes are auto-rewritten to the <code>psycopg 3</code> driver. Unreachable → SQLite fallback <code>uxo_local.db</code> .
Host / port	<code>config.py</code>	<code>0.0.0.0:8000</code>	Also hard-coded in <code>server.py</code> 's <code>__main__</code> block.
API_BASE	<code>frontend/src/App.tsx</code>	<code>http://localhost:8000</code>	Hard-coded; change before deploying anywhere else.
Postgres credentials	<code>docker-compose.yml</code>	<code>postgres / postgres / nolte_geoservices</code>	Image <code>postgis/postgis:16-3.4</code> ; data persisted in Docker volume <code>feedbackapp_postgres_data</code> .
pgAdmin	<code>docker-compose.yml</code>	<code>admin@nolte-geoservices.com / admin, port 8080</code>	Image <code>dpape/pgadmin4:latest</code> ; settings persisted in volume <code>feedbackapp_pgadmin_data</code> .

16. Troubleshooting

“database files are incompatible with server” on `docker compose up`

```
FATAL: database files are incompatible with server
DETAIL: The data directory was initialized by PostgreSQL version 16,
       which is not compatible with this version 15.x
```

Cause: the Docker volume `feedbackapp_postgres_data` keeps the database files between runs, and they are stamped with the PostgreSQL major version that created them. If the `image:` tag in `docker-compose.yml` points to an *older* major version than the volume's `PG_VERSION`, PostgreSQL refuses to start.

Fix (non-destructive, preferred): set the image to the major version that matches the data — currently `postgis/postgis:16-3.4` — and `docker compose up -d` again. You can check what version the volume holds with:

```
docker run --rm -v feedbackapp_postgres_data:/data alpine cat /data/PG_VERSION
```

Fix (destructive): if the data is disposable, delete the volume and let the pinned image re-initialize it, then re-ingest: `docker compose down -v` (erases both database and pgAdmin volumes).

“Found orphan containers” warning

Compose prints this when a container labelled with this project (folder name `feedbackapp`) exists but is not declared in the current compose file. It is harmless. It occurred here for `nolte-pgadmin` before pgAdmin was added to the compose file; with both services declared, the warning is gone. Only use `--remove-orphans` if you actually want such containers deleted.

Backend silently uses SQLite instead of PostgreSQL

If PostgreSQL is unreachable for 5 seconds at startup, `database.py` logs an error and falls back to `sqlite:///./uxo_local.db`. The app keeps working, but you are no longer writing to the shared database — the map may look empty or stale. Check the server startup log for “Falling back

to local SQLite database”, make sure the Docker container is up, then restart `python server.py` .

Sync fails with HTTP 400 “database has not been seeded with targets yet”

`POST /api/sync` returns this when the commit fails — most commonly because feedback references anomalies that don't exist in the current database (e.g. the DB was wiped or you are on the SQLite fallback). Load targets first (`ingest_anomalies.py` or `POST /api/seed`), then sync again; queued items are preserved in the browser until a sync succeeds.

Map shows no points after login

Usual causes, in order: the backend isn't running on port 8000 (the frontend's `API_BASE` is hard-coded to `http://localhost:8000`); the database is empty (seed or ingest); or active filters (project / instrument / status) exclude everything — reset them to “All”.

17. Known Limitations & Notes for Future Development

- **No real authentication.** Login is client-side theatre: passwords are never checked, roles live in localStorage, and every API endpoint is unauthenticated. A real deployment needs server-side auth (e.g. JWT/OAuth) and role enforcement.
- **Open CORS** (`allow_origins=["*"]`) and a hard-coded `API_BASE` pointing at localhost.
- **Destructive endpoints are exposed:** anyone who can reach the server can call `POST /api/seed` and wipe all data.
- **Photos as base64 in the database** will bloat rows quickly; object storage (files/S3) with references would scale better.
- **N+1 query pattern:** `GET /api/points` runs one feedback query per anomaly — fine for hundreds of targets, slow for the full ~2,400-target magnetic dataset; a join or eager-load would fix it.
- **Keyword-based UXO detection** (searching notes for “mine”, “bomb”, ...) is fragile — a note like “nothing resembling a mine” would flag as UXO. An explicit status field chosen by the investigator would be more reliable.
- **Single hard-coded project** (`11-24-2736`) in import/seed paths; multi-project support is scaffolded (project dropdown, projects table) but not complete.
- **Duplicated conversion math** in Python, TypeScript (twice) and SQL — four implementations to keep consistent; the Python/TS versions also ignore the small geoid differences a library like `pyproj` / `proj4js` would handle.
- **Git hygiene:** compiled `__pycache__` files and local DB files are tracked/untracked inconsistently; a `.gitignore` would help. The `node/` folder (two full Node distributions) makes the repo heavy.